# Отчёт по лабораторной работе №1

Курс «Облачные технологии»

nginx как точка входа и балансировщик перед сервисом «Заметки»

Выполнили: Романов Егор, Марьёв Егор

## 1. Цель

Сделать из nginx точку входа и балансировщик перед сервисом из лабы 0 и добиться отказоустойчивости: сайт должен продолжать отвечать, когда один из двух бэкендов упал.

## 2. Репозиторий с кодом

Ссылка: https://github.com/ERRomanov/CloudMonday/tree/main/cloud-lab1

## 3. Архитектура

Бэкенд — Flask-приложение из лабы 0. Его две копии запускаются на портах 5001 и 5002 с разными идентификаторами (`INSTANCE_ID`). Каждый ответ содержит заголовок `X-Instance-Id`, а `/api/health` возвращает поле `instance`, поэтому видно, какая копия ответила.

| Требование | Как выполнено в nginx.conf |
|---|---|
| HTTPS с самоподписанным сертификатом | `listen 443 ssl`, `ssl_certificate`, `ssl_certificate_key` |
| HTTP (80) -> HTTPS (443) | `return 301 https://$host$request_uri;` |
| Статику отдаёт nginx, `/api` проксируется | `root frontend;`, `location /api/ { proxy_pass http://backend_pool; }` |
| Балансировка между двумя копиями | `upstream backend_pool` с серверами :5001 и :5002 |
| Отказоустойчивость | `max_fails=1 fail_timeout=5s`, `proxy_next_upstream`, `proxy_connect_timeout 1s` |
| `alias` | `location /docs/ { alias docs/; }` |
| Виртуальные хосты | `server_name lab.local`, `second.local`, `default_server` для чужих имён |
| Ограничение частоты | `limit_req_zone ... rate=5r/s`, `limit_req zone=api burst=10 nodelay`, статус 429 |
| Своя 404 (и 50x) | `error_page 404 /404.html;`, `error_page 500 502 503 504 /50x.html;` |
| `/admin` под паролем | `auth_basic` + `auth_basic_user_file htpasswd` |

## 4. Конфигурация nginx

Файл `nginx/nginx.conf` целиком:

```
# Лаба 1. nginx как точка входа и балансировщик перед сервисом «Заметки».
# # Запуск (из корня
лабы):
#   nginx -p "$PWD/" -c nginx/nginx.conf -e logs/error.log
# # Относительные
пути:
```

```
#   root / alias / error_page / логи  — от каталога, заданного ключом -p (корень
лабы); #   ssl_certificate / htpasswd / include — от каталога этого файла (nginx/).

worker_processes 1; pid
run/nginx.pid; error_log
logs/error.log warn;

events {
worker_connections 1024; }

http {     include       mime.types;
default_type  application/octet-stream;
charset       utf-8;     sendfile
on;     server_tokens off;

    # Временные файлы держим внутри проекта (так конфиг переносим между
ОС).     client_body_temp_path run/client_body;     proxy_temp_path
run/proxy;     fastcgi_temp_path     run/fastcgi;     uwsgi_temp_path
run/uwsgi;     scgi_temp_path        run/scgi;

    # В лог пишем, какой бэкенд обработал запрос: видно балансировку.
log_format main '$remote_addr [$time_local] $host "$request" $status '
'upstream=$upstream_addr instance=$upstream_http_x_instance_id';     access_log
logs/access.log main;

    # ------------------------------------------------------------
--
    # Ограничение частоты запросов: 5 запросов/сек с одного IP,
# всплеск до 10 запросов, дальше — ответ 429.     # ---------------
-----------------------------------------------    limit_req_zone
$binary_remote_addr zone=api:10m rate=5r/s;     limit_req_status
429;

    # ------------------------------------------------------------
-    # Балансировка: два экземпляра бэкенда на разных портах.
    # Если экземпляр не отвечает, nginx 5 секунд не шлёт ему
запросы.     # ----------------------------------------------------
----------    upstream backend_pool {         server 127.0.0.1:5001
max_fails=1 fail_timeout=5s;         server 127.0.0.1:5002
max_fails=1 fail_timeout=5s;     }

    # ------------------------------------------------------------
-    # HTTP (80): всё перенаправляем на HTTPS.     # --------------
-----------------------------------------------    server {
        listen 80 default_server;
server_name _;

        return 301 https://$host$request_uri;
}

    # ------------------------------------------------------------
-    # HTTPS: сервер по умолчанию. Сюда попадают запросы с чужим
Host:
    # ничего из наших проектов он не отдаёт.     # ----------------
-----------------------------------------------    server {
        listen 443 ssl default_server;
server_name _;

        ssl_certificate     certs/server.crt;
ssl_certificate_key certs/server.key;         default_type
text/plain;         return 404 "Unknown host\n";     }

    # ------------------------------------------------------------
-    # Виртуальный хост 1: основной проект (фронт + API).     # ---
-----------------------------------------------------------
server {         listen 443 ssl;         server_name lab.local;

        ssl_certificate     certs/server.crt;
ssl_certificate_key certs/server.key;
ssl_protocols       TLSv1.2 TLSv1.3;         add_header
X-Content-Type-Options nosniff always;

        # Статику фронтенда отдаёт сам nginx.
```

```
        root  frontend;
index index.html;

        # Свои страницы ошибок.         error_page
404 /404.html;         error_page 500 502 503 504
/50x.html;         location = /404.html { root
errors; internal; }         location = /50x.html {
root errors; internal; }

        location / {
            try_files $uri $uri/ =404;
}

        # /api -> один из двух бэкендов.
        location /api/ {             limit_req
zone=api burst=10 nodelay;

            proxy_pass http://backend_pool;             proxy_set_header
Host              $host;             proxy_set_header X-Real-IP
$remote_addr;             proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;             proxy_set_header X-Forwarded-
Proto $scheme;

            # Отказоустойчивость: упавший экземпляр быстро
отсекается,             # запрос повторяется на соседнем.
proxy_connect_timeout 1s;

            proxy_next_upstream error timeout http_502 http_503 http_504;
proxy_next_upstream_tries 2;

            add_header X-Upstream $upstream_addr
always;         }

        # alias: URL /docs/ отдаётся из отдельной папки docs/.
        location = /docs { return 301 /docs/;
}         location /docs/ {             alias
docs/;             index index.html;
}

        #  /admin  закрыт  базовой  авторизацией.
location  =  /admin  {  return  301  /admin/;  }
location /admin/ {                   auth_basic
"Admin area";               auth_basic_user_file
htpasswd;           alias admin/;           index
index.html;         }
    }

    #  ------------------------------------------------------------
-    # Виртуальный хост 2: второй «проект» — просто статика.     # -
------------------------------------------------------------
server {          listen 443 ssl;          server_name second.local;
ssl_certificate     certs/server.crt;
        ssl_certificate_key certs/server.key;
ssl_protocols       TLSv1.2 TLSv1.3;

        root  second;
index index.html;

        error_page 404 /404.html;         location =
/404.html { root errors; internal; }

        location / {
try_files $uri $uri/ =404;         }
    }
}
```

## 5. Процесс настройки

Порядок настройки:

1. Сгенерирован самоподписанный сертификат для `lab.local`, `second.local` и `localhost` (скрипт `gen-cert.sh`, команда `openssl req -x509`). Параметры сертификата:

```
$ openssl x509 -in nginx/certs/server.crt -noout -subject -dates -ext subjectAltName
subject=CN = lab.local notBefore=Oct 30 23:55:26 2026 GMT notAfter=Oct 30 23:55:26
2027 GMT X509v3 Subject Alternative Name:
    DNS:lab.local, DNS:second.local, DNS:localhost

$ cat nginx/htpasswd   (хеш пароля, формат apr1) admin:$apr1$...
```

2. Создан файл паролей для `/admin` командой `htpasswd -cbm nginx/htpasswd admin ПАРОЛЬ` (хеш в формате apr1, подходит и для nginx на Windows).
3. В файл hosts добавлена строка `127.0.0.1 lab.local second.local`.
4. Запущены две копии бэкенда на портах 5001 и 5002 (`./start-backends.sh`).
5. Запущен nginx из корня лабы; пути в конфиге считаются от ключа `-p`:




*Рис. 1. Запуск nginx с конфигом лабы (PowerShell); окно остаётся открытым, пока nginx работает* Проверка конфигурации (ключ `-t`):

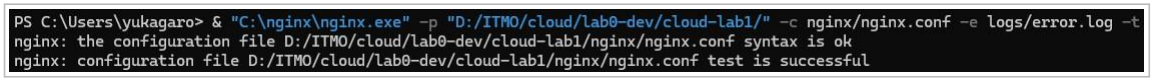



*Рис. 2. nginx -t: синтаксис конфига корректен*

## 6. Скрипт проверок check.sh

Скрипт проверяет каждый пункт задания и в конце печатает итог. Исходный код:

```
#!/usr/bin/env bash # Проверки лабы 1. Каждый пункт задания проверяется
отдельно, в конце — итог.
# Запуск:  ./check.sh | tee check-output.txt
# # Предусловия: запущены оба бэкенда (./start-backends.sh) и nginx (sudo ./start-
nginx.sh).
# Имена lab.local и second.local подставляются через curl --resolve, поэтому
# править файл hosts для этого скрипта не нужно. cd
"$(dirname "$0")"

HOST=lab.local
OTHER=second.local
FOREIGN=evil.example
ADMIN_USER=admin
ADMIN_PASS=lab1pass

# curl: -k (самоподписанный сертификат), без системного прокси, имена -> 127.0.0.1
c() {   curl -sk --noproxy '*' \     --resolve "$HOST:80:127.0.0.1"  --resolve
"$HOST:443:127.0.0.1" \
    --resolve "$OTHER:443:127.0.0.1" --resolve "$FOREIGN:443:127.0.0.1" "$@"
}

PASS=0; FAIL=0 ok()   { echo "  [PASS] $1"; PASS=$((PASS + 1)); } bad()  { echo "  [FAIL] $1"; FAIL=$((FAIL + 1));
} check() { if [ "$1" = "1" ]; then ok "$2"; else bad "$2"; fi; } instance() { c -D - -o /dev/null "$1" | tr -d
'\r' | awk -F': ' 'tolower($1)=="x-instance-id"{print $2}'; } # probe URL -> "КОД ИДЕНТИФИКАТОР" одним запросом
(чтобы не расходовать лимит запросов) probe() { c -D - -o /dev/null -w 'CODE=%{http_code}\n' "$1" | tr -d '\r' |
awk -F': ' 'tolower($1)=="x-instance    id"{i=$2} /^CODE=/{sub("CODE=","");c=$0} END{print c, i}'; } title() {
echo; echo "=== $1"; }

# ------------------------------------------------------------------------title "1. HTTP -> HTTPS: редирект
301" echo "\$ curl -I http://$HOST/api/health?x=1" c -sI "http://$HOST/api/health?x=1" | tr -d '\r' | grep -i
-E "^HTTP|^Server|^Location" code=$(c -o /dev/null -w '%{http_code}' "http://$HOST/") loc=$(c -o /dev/null -w
```

```
'%{redirect_url}' "http://$HOST/api/health?x=1") check "$([ "$code" = 301 ] && echo 1)" "HTTP отвечает 301
(получено $code)" check "$([ "$loc" = "https://$HOST/api/health?x=1" ] && echo 1)" "Location ведёт на https и
сохраняет путь и     параметры ($loc)" if [ "$code" != 301 ]; then   echo "  подсказка: порт 80 отвечает не
наш nginx (в заголовках выше должно быть 'Server: nginx')."   echo "  Посмотрите, кто слушает порт 80:
netstat -ano | findstr :80   (Windows)"   echo "                                    sudo ss -ltnp | grep ':80
'   (Linux)" fi

# ------------------------------------------------------------------------title "2.
Балансировка: запросы чередуются между двумя экземплярами" echo "\$ for i in 1..6; do curl -k
https://$HOST/api/health; done  (заголовок X-Instance-Id)" seen="" for i in 1 2 3 4 5 6; do
inst=$(instance "https://$HOST/api/health")   echo "  запрос $i -> $inst"   seen="$seen $inst"
done echo "  тело ответа: $(c "https://$HOST/api/health")"
check "$(echo "$seen" | grep -q backend-1 && echo "$seen" | grep -q backend-2 && echo 1)" "ответили оба экземпляра
(backend-1 и backend-2)"

# ------------------------------------------------------------------------
title "3. Отказоустойчивость: гасим backend-2, сайт продолжает отвечать" sleep
2 ./stop-backends.sh 2 sleep 1 echo "\$ for i in 1..8; do curl -k
https://$HOST/api/health; done" codes=""; seen="" for i in 1 2 3 4 5 6 7 8; do
  read -r code inst <<< "$(probe "https://$HOST/api/health")"
echo "  запрос $i -> HTTP $code, ответил $inst"
codes="$codes $code"; seen="$seen $inst" done
check "$([ -z "$(echo "$codes" | tr ' ' '\n' | grep -v -x -e 200 -e '')" ] && echo 1)" "все ответы 200 при одном
упавшем бэкенде" check "$(echo "$seen" | grep -q backend-2 || echo 1)" "трафик ушёл только на живой backend-1"
sleep 2 echo "  записи на сайте при упавшем backend-2 тоже работают:" # Тело запроса идёт через stdin: так кавычки
JSON не искажаются командной строкой Windows. code=$(printf '%s' '{"text":"write while backend-2 is down"}' | c -o
/dev/null -w '%{http_code}' -X POST -H     'Content-Type: application/json' --data-binary @-
"https://$HOST/api/notes") check "$([ "$code" = 201 ] && echo 1)" "POST /api/notes -> $code"

echo "  поднимаем backend-2 обратно"
./start-backends.sh 2 > /dev/null
sleep 6   # fail_timeout=5s: после него nginx снова начинает слать запросы на backend-2 seen="" for i in 1 2 3
4 5 6; do seen="$seen $(instance "https://$HOST/api/health")"; done check "$(echo "$seen" | grep -q backend-2
&& echo 1)" "backend-2 снова получает трафик после восстановления"

# ------------------------------------------------------------------------title "4. /admin
закрыт базовой авторизацией" echo "\$ curl https://$HOST/admin/" code=$(c -o /dev/null -w
'%{http_code}' "https://$HOST/admin/") check "$([ "$code" = 401 ] && echo 1)" "без пароля ->
$code" code=$(c -o /dev/null -w '%{http_code}' -u "$ADMIN_USER:wrong" "https://$HOST/admin/")
check "$([ "$code" = 401 ] && echo 1)" "с неверным паролем -> $code" code=$(c -o /dev/null -w
'%{http_code}' -u "$ADMIN_USER:$ADMIN_PASS" "https://$HOST/admin/") check "$([ "$code" = 200
] && echo 1)" "с верным паролем -> $code"

# ------------------------------------------------------------------------title "5. Ограничение частоты: флуд на
/api упирается в 429" sleep 3   # даём «ведру» limit_req наполниться echo "\$ 60 запросов подряд на
https://$HOST/api/health" stats=$(for i in $(seq 1 60); do c -o /dev/null -w '%{http_code}\n'
"https://$HOST/api/health"; done | sort | uniq     -c) echo "$stats" | sed 's/^/  /' n429=$(echo "$stats" | awk
'$2==429{print $1}') check "$([ "${n429:-0}" -gt 0 ] && echo 1)" "получено ответов 429: ${n429:-0}" sleep 3

# ------------------------------------------------------------------------
title "6. Виртуальные хосты: каждый домен отдаёт только свой проект" body=$(c
"https://$HOST/")
check "$(echo "$body" | grep -q 'Заметки' && ! echo "$body" | grep -q SECOND-PROJECT-MARKER && echo 1)" "Host: $HOST
-> основной проект, без чужого содержимого" body=$(c "https://$OTHER/") check "$(echo "$body" | grep -q SECOND-
PROJECT-MARKER && ! echo "$body" | grep -q 'Заметки' && echo 1)" "Host:     $OTHER -> второй проект, без содержимого
основного" code=$(c -o /dev/null -w '%{http_code}' "https://$OTHER/api/health") check "$([ "$code" = 404 ] && echo 1)"
"у $OTHER нет API ($code), чужие маршруты не доступны"

body=$(c -w ' [HTTP %{http_code}]' "https://$FOREIGN/") echo "  Host: $FOREIGN -> $(echo "$body" | tr '\n' ' ')" check
"$(echo "$body" | grep -q 'HTTP 404' && ! echo "$body" | grep -q -e 'Заметки' -e SECOND-PROJECT-MARKER && echo
1)" "чужой Host -> 404, ни один из проектов не отдан"

# ------------------------------------------------------------------------title "7. Своя страница 404"
echo "\$ curl https://$HOST/no-such-page" body=$(c -w ' [HTTP %{http_code}]' "https://$HOST/no-such-
page") check "$(echo "$body" | grep -q 'HTTP 404' && echo "$body" | grep -q 'Страница не найдена' &&
echo 1)"     "несуществующая страница -> наша 404, а не стандартная nginx"

# ------------------------------------------------------------------------title "8. alias: /docs/
отдаётся из отдельной папки docs/" body=$(c "https://$HOST/docs/") check "$(echo "$body" | grep -q DOCS-
MARKER && echo 1)" "https://$HOST/docs/ -> содержимое папки docs/"

# ------------------------------------------------------------------------
title "9. (по желанию) Оба бэкенда упали -> своя страница 50x" ./stop-
backends.sh > /dev/null sleep 1 body=$(c -w ' [HTTP %{http_code}]'
"https://$HOST/api/health")
echo "  ответ: $(echo "$body" | grep -o -E 'Сервис временно недоступен|\[HTTP [0-9]+\]' | paste -sd' ')" check
"$(echo "$body" | grep -q 'Сервис временно недоступен' && echo 1)" "вместо стандартной ошибки nginx показана
наша страница 50x" ./start-backends.sh > /dev/null sleep 6
```

```
# ------------------------------------------------------------------------
echo echo "================================================" echo "Итого:
PASS=$PASS, FAIL=$FAIL"
[ "$FAIL" -eq 0 ]
```

## 7. Вывод проверок

Запуск в Git Bash: `./check.sh`. Итог: все 18 проверок пройдены (`PASS=18, FAIL=0`).

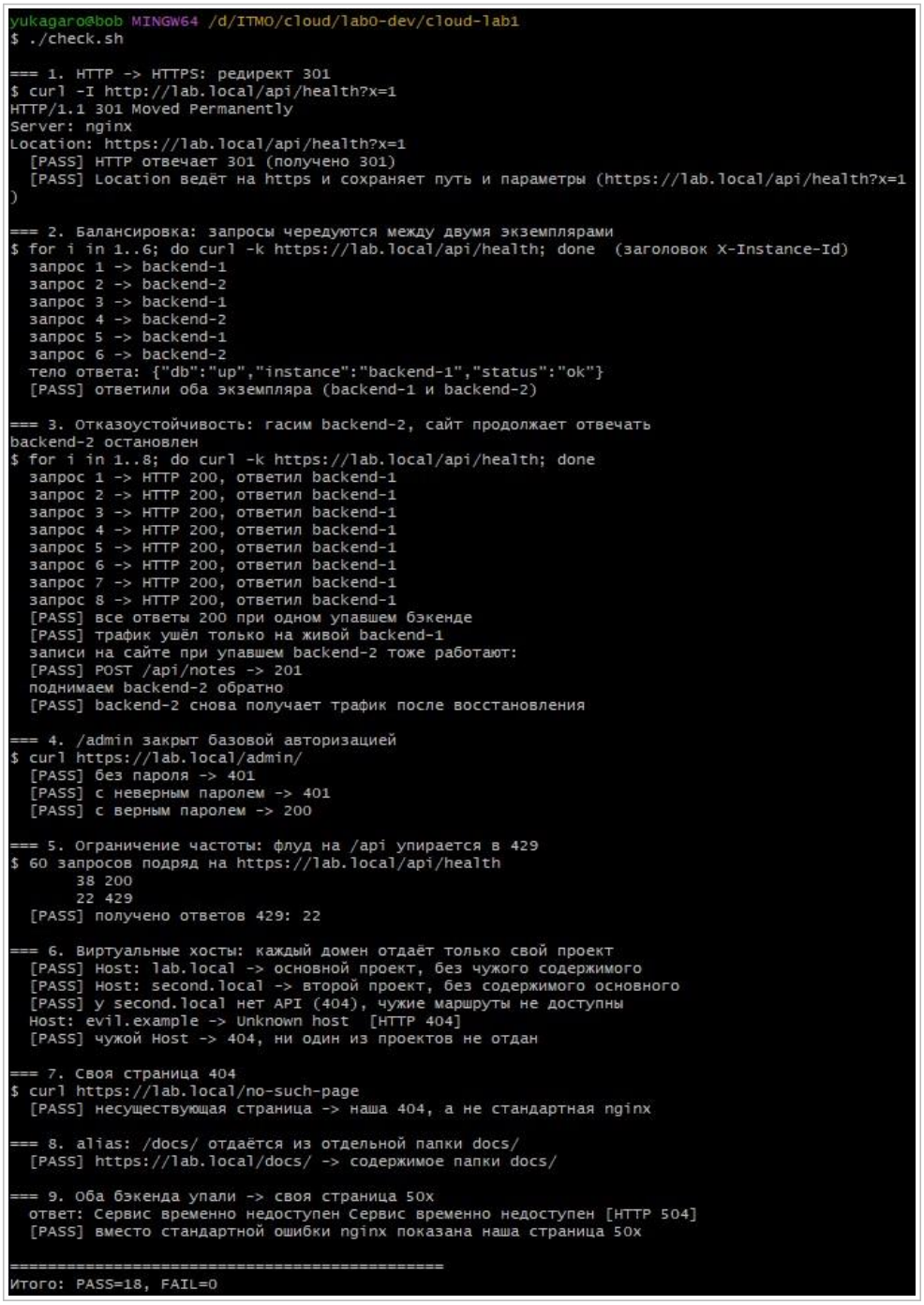



*Рис. 3. Вывод check.sh: редирект 301, чередование backend-1 и backend-2, 200 при упавшем backend-2, 401 на /admin, 429 при флуде, разведение виртуальных хостов, своя 404 и 50x*

## 8. Скриншоты

**Балансировка.** Основная страница по HTTPS (`https://lab.local`). Строка статуса показывает, какой экземпляр бэкенда ответил; она обновляется каждые 3 секунды. На двух снимках ответили разные экземпляры, то есть запросы действительно распределяются между обоими бэкендами.







*Рис. 4. Ответил backend-2* *Рис. 5. Чуть позже ответил backend-1* **Отказоустойчивость.** Справа в окне терминала видно, какой бэкенд остановлен.

Когда остановлен backend-2, сайт продолжает работать, а отвечает backend-1:

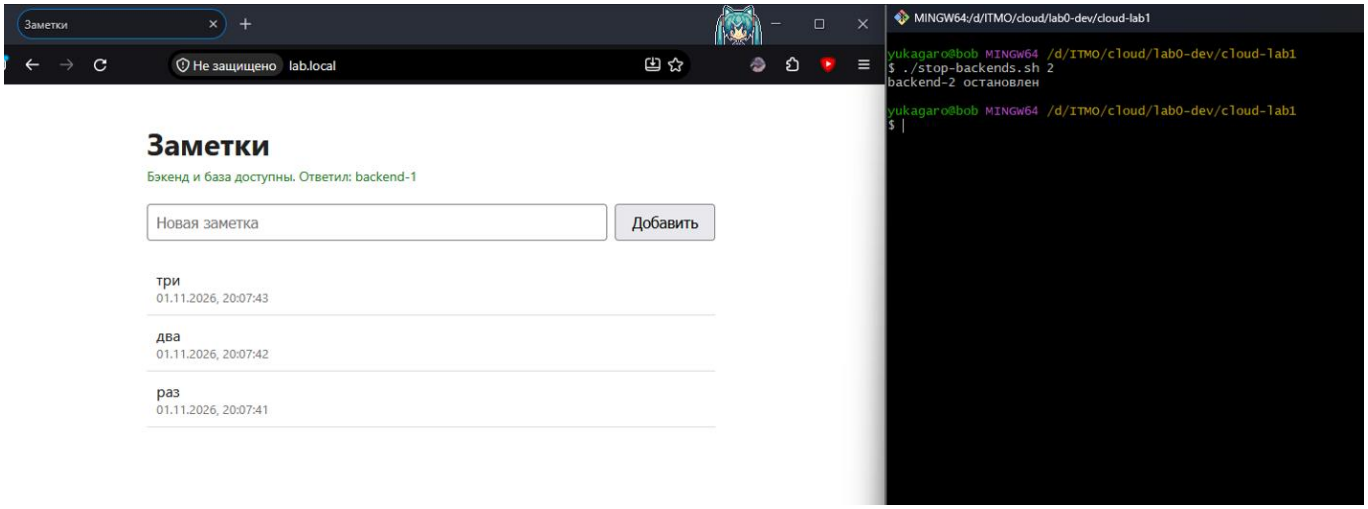



*Рис. 6. backend-2 остановлен: страница работает, ответил backend-1*

И наоборот: backend-2 запущен снова, а остановлен backend-1. Сайт по-прежнему работает, теперь отвечает backend-2:

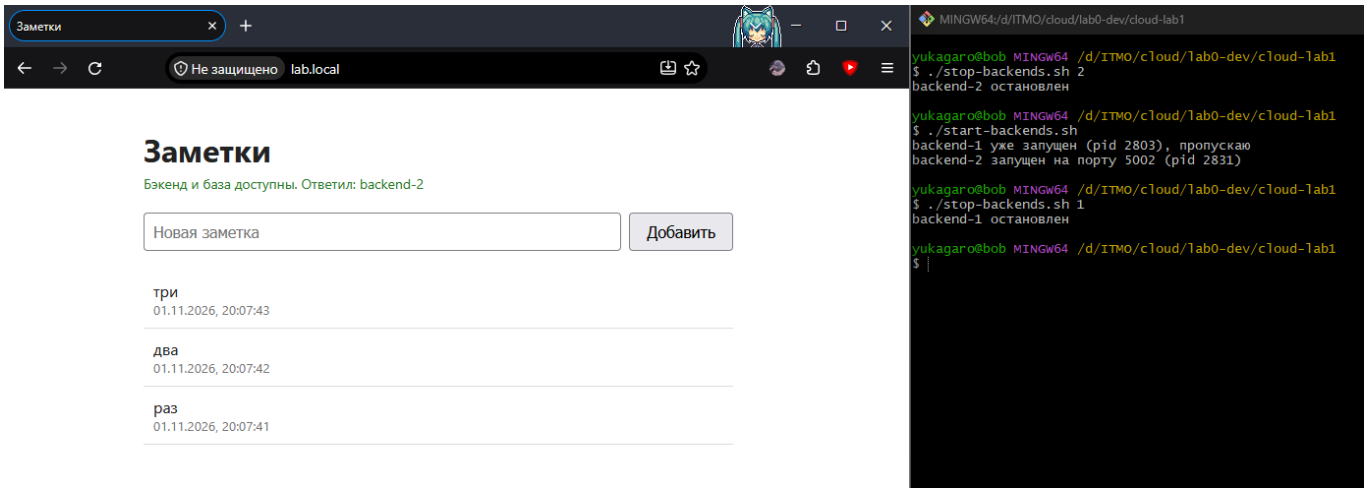



*Рис. 7. backend-1 остановлен: страница работает, ответил backend-2*

**Оба бэкенда остановлены.** nginx продолжает отдавать страницу, а запросы к API завершаются ошибкой, которую страница показывает, как «Бэкенд недоступен»:

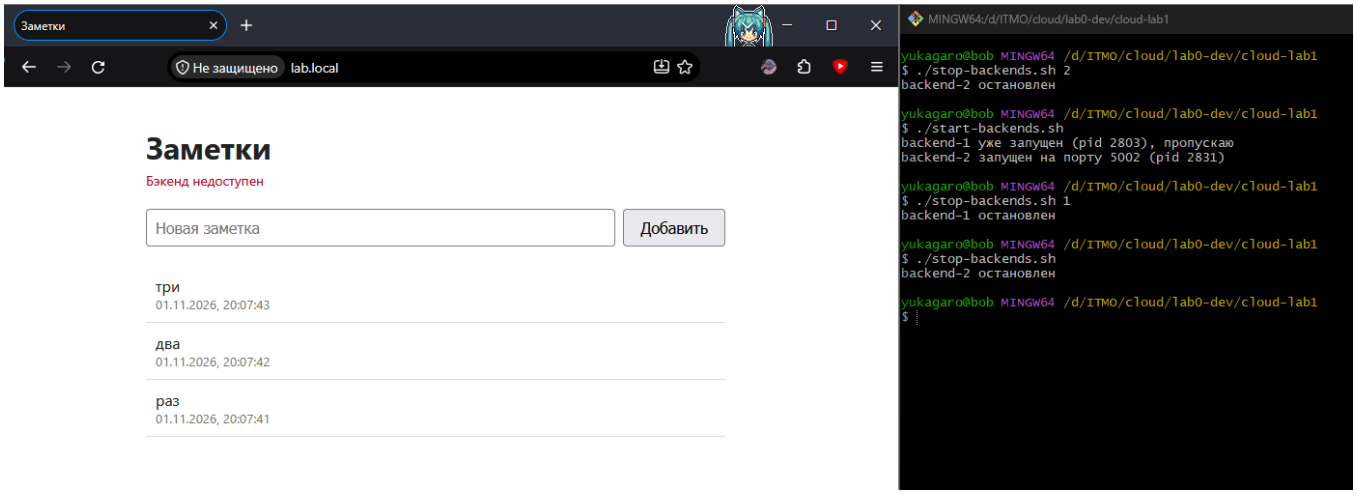



*Рис. 8. Оба бэкенда остановлены: страница «Бэкенд недоступен»*

При прямом обращении к API пользователь видит нашу страницу 50x вместо стандартной ошибки nginx:

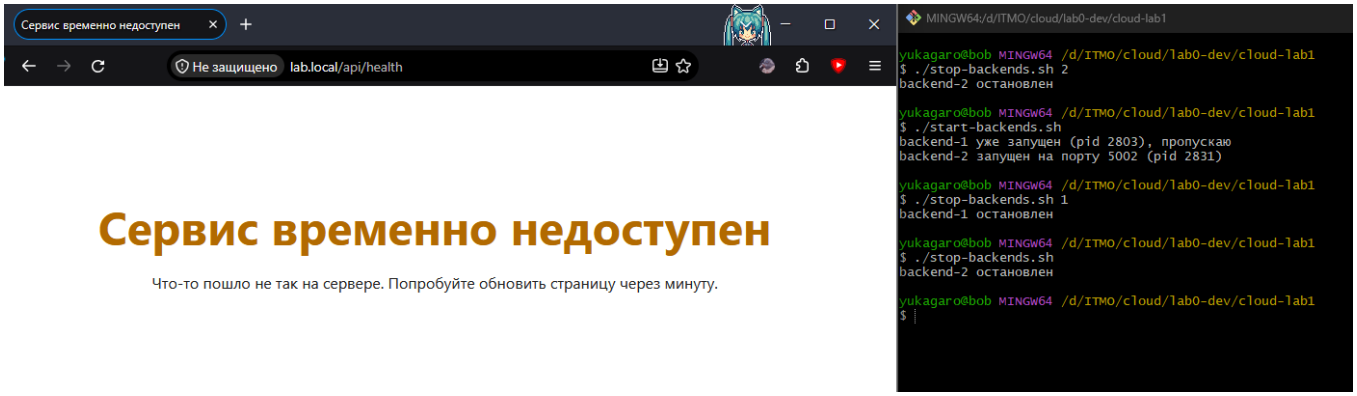



*Рис. 9. https://lab.local/api/health при остановленных бэкендах: своя страница 50x*

**alias, админка, виртуальные хосты, 404.**




*Рис. 11. /admin/ после ввода логина и пароля Рис. 10. /docs/ (alias на папку docs/)*

*Рис. 12. Второй виртуальный хост second.local*

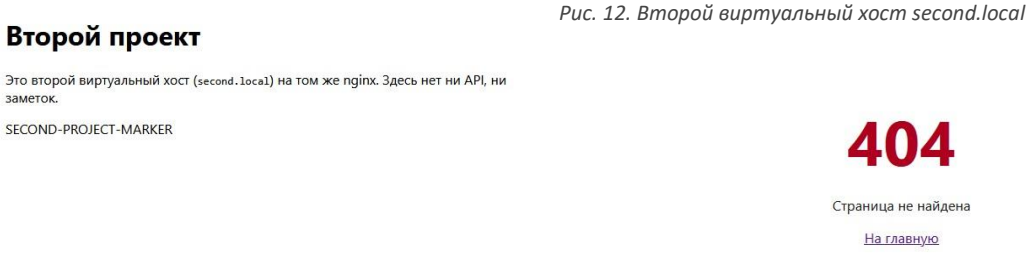



*Рис. 13. Своя страница 404*

Без логина и пароля `/admin/` отвечает 401 (проверка 4 в выводе check.sh на рис. 3): браузер в таком случае показывает системное окно ввода пароля.

## 9. Вывод

nginx настроен как единая точка входа: принудительный HTTPS, статика фронтенда, балансировка `/api` между двумя копиями бэкенда, ограничение частоты запросов, виртуальные хосты, `alias`, свои страницы 404 и 50x, базовая авторизация для `/admin`. Главный результат достигнут: при остановке любого из двух бэкендов сайт продолжает отвечать с кодом 200, запросы автоматически уходят на живой экземпляр, а после восстановления упавшая копия снова получает трафик. Все пункты подтверждены скриптом `check.sh` (18 из 18).

## 10. Проверка уязвимостей (часть со звездочкой)

Попытался найти что-то интересное на сайте горнолыжного курорта города Кировск. Сперва через alias'ы попытался найти открытые файлы, но безуспешно

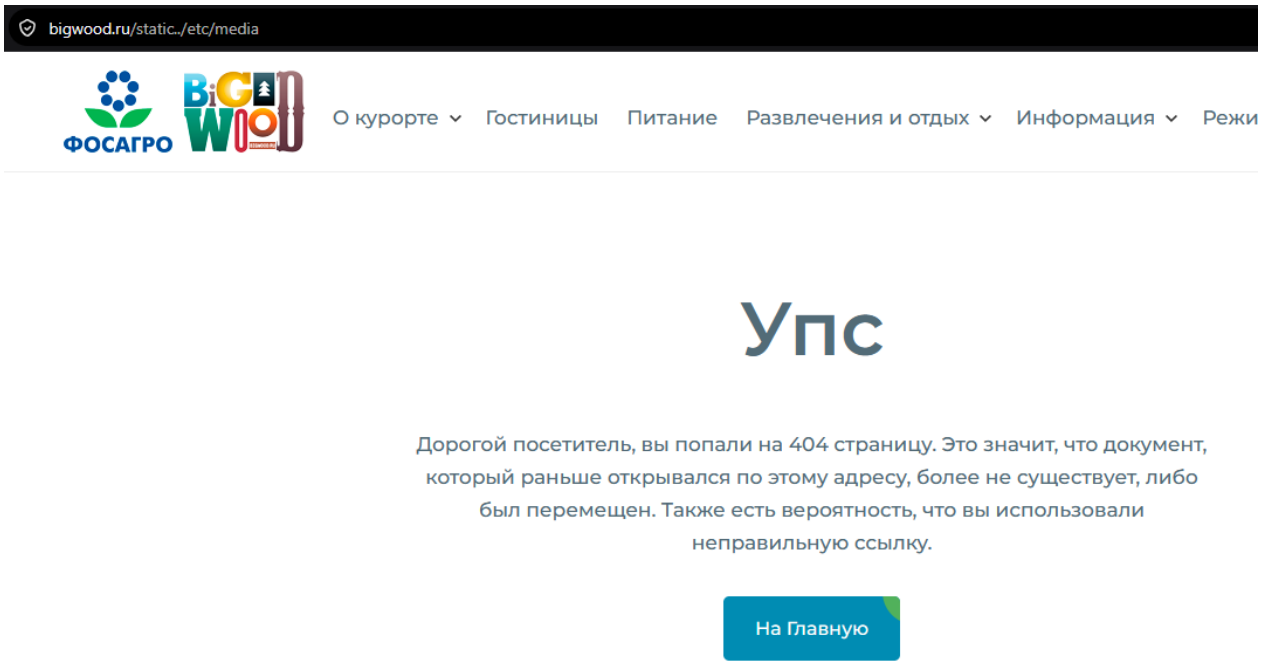



*Рис. 14. Пример запроса файлов медиа в static*

Затем через ffuf перебрал страницы:

```
PS C:\Users\yukagaro> ffuf -u https://bigwood.ru/FUZZ -w "C:\SecLists\Discovery\Web-Content\common.txt" -ac -fc 301

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.3.0
________________________________________________

 :: Method           : GET
 :: URL              : https://bigwood.ru/FUZZ
 :: Wordlist         : FUZZ: C:\SecLists\Discovery\Web-Content\common.txt
 :: Follow redirects : false
 :: Calibration      : true
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
 :: Filter           : Response status: 301
________________________________________________

admin                   [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 783ms]: 0 :::
dashboard               [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 860ms]s: 0 ::
favicon.ico             [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 752ms]s: 0 ::
login                   [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 799ms]s: 0 ::
phpmyadmin              [Status: 403, Size: 2641, Words: 168, Lines: 117, Duration: 16ms]:
robots.txt              [Status: 200, Size: 353, Words: 35, Lines: 17, Duration: 10ms]0 ::
server-status           [Status: 403, Size: 2641, Words: 168, Lines: 117, Duration: 21ms]:
xmlrpc.php              [Status: 403, Size: 2641, Words: 168, Lines: 117, Duration: 18ms]:
:: Progress: [4752/4752] :: Job [1/1] :: 48 req/sec :: Duration: [0:01:41] :: Errors: 0 ::
PS C:\Users\yukagaro>
```

*Рис. 15 Результат ffuf по SecLists*

Проверив куда сайт перенаправляет по https://bigwood.ru/login, стало ясно, что сайт сделан на WordPress, так как перенаправление идет на wp-login.php.

```
PS C:\Users\yukagaro> curl.exe -I https://bigwood.ru/login
HTTP/1.1 302 Found
Server: ddos-guard
Connection: keep-alive
Keep-Alive: timeout=60
Set-Cookie: __ddg8_=CPorVvvJ0hKcGaFN; Domain=.bigwood.ru; Path=/; Expires=Sun, 01-Nov-2026 19:30:02 GMT
Set-Cookie: __ddg10_=1793560202; Domain=.bigwood.ru; Path=/; Expires=Sun, 01-Nov-2026 19:30:02 GMT
Set-Cookie: __ddg9_=185.119.0.121; Domain=.bigwood.ru; Path=/; Expires=Sun, 01-Nov-2026 19:30:02 GMT
Content-Security-Policy: upgrade-insecure-requests;
Set-Cookie: __ddg1_=vQyY9FqLRsqYnHasBioP; Domain=.bigwood.ru; HttpOnly; Path=/; Expires=Mon, 01-Nov-2027 19:10:02 GMT
Date: Sun, 01 Nov 2026 19:10:02 GMT
Content-Type: text/html; charset=UTF-8
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0, no-store, private
X-Redirect-By: WordPress
Location: https://bigwood.ru/wp-login.php
Strict-Transport-Security: max-age=31536000;
```

*Рис. 16 Перенаправление на wp-login.php*

Однако при попытке перейти по запросу выкидывало 403 страницу, уязвимости здесь нет.

Проверив остальные страницы результат был тот же, кроме robots.txt. Он же доступен по адресу https://bigwood.ru/robots.txt, и показывает скрытые или служебные пути. Кроме того в файле явно разрешен доступ к скрипту admin-ajax.php.

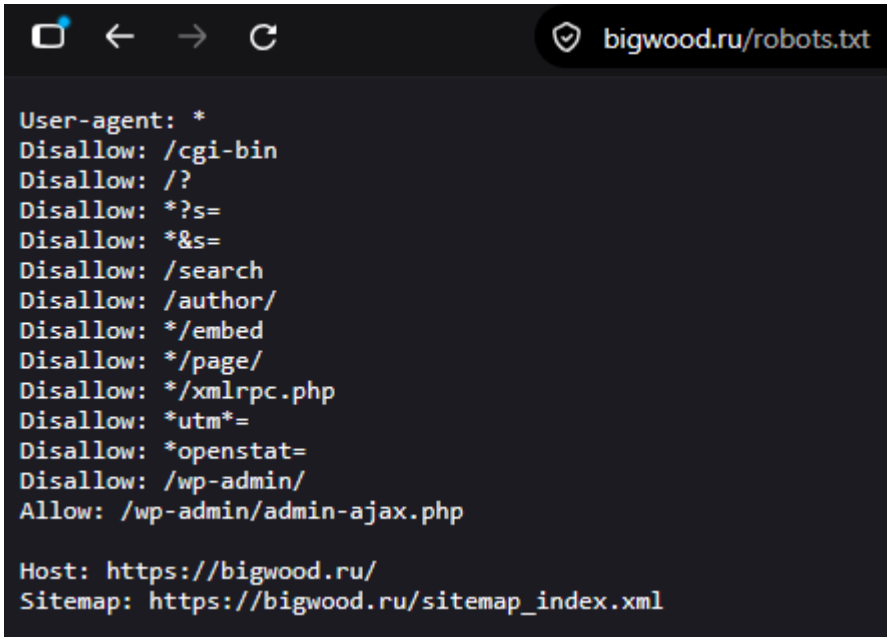



*Рис. 17 Содержимое https://bigwood.ru/robots.txt*

Как итог, найти какой-то уязвимости не получилось :C